Efficient Exhaustive Verification of the Collatz Conjecture using DSP48E blocks of Xilinx Virtex-5 FPGAs

Yasuaki Ito and Koji Nakano
Department of Information Engineering
Hiroshima University
Kagamiyama 1-4-1, Higashi-Hiroshima, 739-8527, JAPAN
{*yasuaki, nakano*}*@cs.hiroshima-u.ac.jp*

***Abstract*—Consider the following operation on an arbitrary positive number: if the number is even, divide it by two, and if the number is odd, triple it and add one. The Collatz conjecture asserts that, starting from any positive number m, repeated iteration of the operations eventually produces the value 1. The main contribution of this paper is to present an efficient implementation of a coprocessor that performs the exhaustive search to verify the Collatz conjecture using a DSP48E Xilinx Virtex-5 blocks, each of which contains one multiplier and one adder. The experimental results show that, our coprocessor can verify 3.88×10^8 64-bit numbers per second.**

***Keywords*-Hardware Algorithm; Collatz conjecture; FPGA Implementation; DSP blocks, block RAMs;**

I. Introduction

An FPGA (Field Programmable Gate Array) is a programmable VLSI in which a hardware designed by users can be embedded instantly. Typical FPGAs consist of an array of programmable logic blocks (or slices), memory blocks, and programmable interconnect between them. The logic block contains four-input logic functions implemented by a look up table and/or several registers. Using four-input logic functions, registers, and their interconnects, any combinational circuits and sequential logic can be implemented. Using design tools provided by FPGA vendors or third party companies, a hardware logic designed by users using hardware description languages can be embedded in FPGAs. Recent FPGAs have a DSP block with a multiplier and an adder, which can performs multiply-accumulate operation in high clock frequency. It has been shown that a lot of computation can be accelerated using a circuit implemented in FPGAs [1]–[5].

The Collatz conjecture is a well-known unsolved conjecture in mathematics [6]–[8]. Consider the following operation on an arbitrary positive number:

even operation if the number is even, divide it by two, and

odd operation if the number is odd, triple it and add one.

The Collatz conjecture assert that, starting from any positive number, repeated iteration of the operations eventually produces the value 1. For example, starting from 3, we have the following sequence to produce 1.

$$3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

The exhaustive verification of the Collatz conjecture is to perform the repeated operations for numbers from 1 to the infinite as follows:

for $m \leftarrow 1$ to ∞
  begin
    $n \leftarrow m$
    while $n \geq m$
    if n is even **then** $n \leftarrow \frac{n}{2}$
    else $n \leftarrow 3n + 1$
  end

Clearly, if the Collatz conjecture is not true, then the while-loop in the program above never terminates for a counter example m. Working projects for the Collatz conjecture are currently checking 60-bit numbers [9] and 63-bit numbers [10].

There are several researches for accelerating the exhaustive verification of the Collatz conjecture. It is known [11] that series of even and odd operations for n can be done in one step by computing $n \leftarrow B[n_L] \cdot n_H + C[n_L]$ for appropriate tables B and C, where the concatenation of n_H and n_L corresponds to n. In [12], [13], FPGA implementations have been presented to repeat the operations of Collatz conjecture. However, these implementations perform the even and odd operations for some fixed size of bits of interim numbers, and ignores the overflow. Hence, if there exists a counter example number m for the Collatz conjecture such that, infinitely large numbers are generated by the operations from m, their implementation may fail to detect it.

In our previous paper [11], we have shown a software-hardware cooperative approach to verify the Collatz conjectures for 64-bit numbers n. Our approach supports almost infinitely large interim numbers m. The idea is to perform the while-loop for interim values with up to 78 bits using a coprocessor embedded in an FPGA. If an interim value m has more than 78 bits, the original value n is reported to the host PC. The host PC performs the verification for such n

978-1-4244-6534-7/10/$26.00 ©2010 IEEE

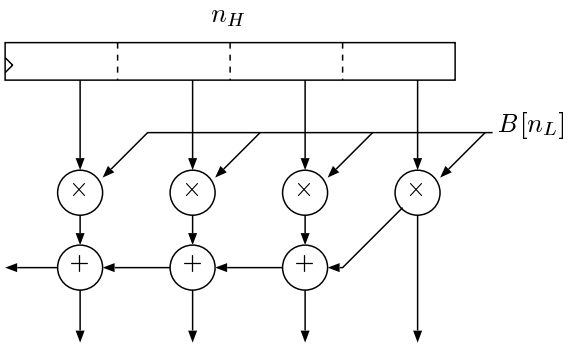

Figure 1. The parallel implementation of 68×16-bit multiplication

using unlimited number of bits by software. This software-hardware cooperative approach makes sense, because

- the hardware implementation on the FPGA is fast and low power consumption, but the number of bits for the operation is fixed,
- the software implementation on the PC is relatively slow and high power consumption, but the number of bits for the operation is unlimited.

In this paper, we improve the coprocessor architecture of our previous paper [11]. We use an embedded DSP block to further accelerate the coprocessor, while four (unsigned) 17-bit multipliers are used in our previous implementation. Let us explain more details of coprocessor architecture of our previous paper [11]. Let n be an interim number, and n_L and n_H denote the least significant 10 bits of n and the remaining bits. The coprocessor performs computation $n \leftarrow B[n_L] \cdot n_H + C[n_L]$ for the exhaustive verification of the Collatz conjecture. We have used embedded (unsigned) 17×17-bit multiplier in Xilinx Virtex-II Family FPGA XC2V3000. As illustrated in Figure 1, we have used *a parallel implementation* that includes four multipliers to compute 68×17-bit multiplication $B[n_L] \cdot n_H$. Based on this idea, we have implemented 24 coprocessors in XC2V3000 which repeatedly perform operation $n \leftarrow B[n_L] \cdot n_H + C[n_L]$ in each of 24 coprocessors in parallel. If the resulting value is overflow, that is, if n_H has more than 68 bits, the value of n is reported in the host PC. Each coprocessor can verify 2.47×10^8 64-bit numbers m per second. The remaining verification is performed for m with overflow interim value n by unbounded bit operations by software on the host PC.

The main contribution of this paper is to further accelerate computation $B[n_L] \cdot n_H + C[n_L] \rightarrow n$ using a DSP block of the FPGA for each coprocessor. More specifically, computation $B[n_L] \cdot n_H$ is performed using a DSP48E block in Xilinx Virtex-5 Family FPGA. Figure 2 illustrates our implementation to compute $n \cdot B[n_L]$ using a DSP48E block. We have six 17-bit registers to store up to 112-bit values of n. The multiplication is computed in a pipeline fashion, and

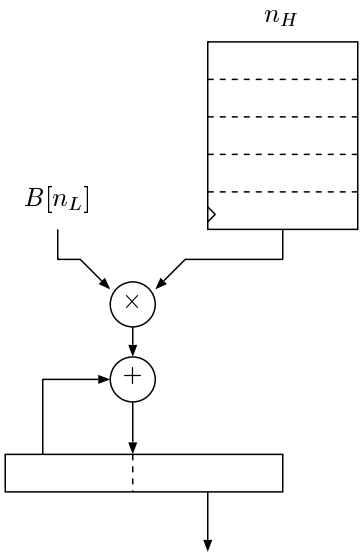

Figure 2. The sequential pipeline implementation

$17k \times 17$bit ($k \leq 6$) multiplication can be performed in k clock cycles.

Our new implementation, *the sequential pipeline implementation* in Figure 2 has advantages over our previous implementation, the parallel implementation in Figure 1. Let us compare the sequential pipeline implementation and the parallel implementation for general case. The sequential pipeline implementation can perform $17k \times 17$-bit multiplication in k clock cycles using a single multiplier for any $k \geq 1$. On the other hand, the parallel implementation performs $17k \times 17$-bit multiplication in 1 clock cycle using k multipliers for a fixed $k \geq 1$. Suppose we have an FPGA with w multipliers and need to multiply of W pairs of $17c$-bit and 17-bit numbers. Using the sequential implementation the pair can be multiplied in c clock cycles. Thus, using w multipliers, w pairs of $17c$-bit and 17-bit numbers can be multiplied in c clock cycles. Hence, W multiplications can be performed in $\lceil \frac{W}{w} \rceil \cdot c \approx \frac{Wc}{w}$ clock cycles. The parallel implementation cannot perform the multiplication whenever $c > k$. If $c \leq k$, then a pair can be multiplied in 1 clock cycles using w multipliers. Thus, the w multipliers can multiply W pairs in $\lceil \frac{W}{\lceil \frac{w}{k} \rceil} \rceil \approx \frac{Wk}{w}$ clock cycles. Also, note that if $c < k$ then $k - c$ multipliers are not used for the multiplication. Hence, the sequential pipeline implementation is more flexible and scalable in the sense that it supports more bits and all multipliers are always used in every clock cycle. Also, since we have carefully used the sequential pipeline implementation, the clock frequency is much higher than the parallel implementation. Our sequential pipeline implementation runs in 280.604MHz, while the parallel implementation runs in 49.520MHz. Also, the overflow is detected if an interim number has more than 78 bits in our

previous parallel implementation, while our new sequential pipeline implementation supports interim number up to 112 bits. Thus, the probability of the overflow, which is reported to the host PC in our new implementation, is much smaller. It follows that the PC needs to verify fewer numbers if we use the sequential pipeline implementation.

Table I summarizes our new implementation and previous implementation. Our parallel implementation [11] uses 4 embedded multipliers and runs in 49.520MHz. Also it can perform the verification for 2.47×10^8 numbers per second using one co-processor. On the other hand, our new sequential pipeline implementation runs in 280.604MHz and it can verify 3.88×10^8 numbers per second using only one DSP48E block, which contains one multiplier.

II. Accelerating the verification of the Collatz conjecture

The main purpose of this section is to introduce a hardware algorithm for accelerating the verification of the Collatz conjecture. The basic ideas of acceleration are shown in [7], [8].

The first technique is to terminate the operations before the iteration produces 1. Suppose that we have already verified that the Collatz conjecture is true for numbers less than n, and we are now in position to verify it for number n. Clearly, if we repeatedly execute the operations for n until the value is 1, then we can confirm that the conjecture is true for n. Instead, if the value becomes n' for some n' less than n, then we can guarantee that the conjecture is true for n because it has been proved to be true for n'. Thus, it is not necessary to repeat this operation until the value is 1, and we can terminate the iteration when, for the first time, the value is less than n.

The second technique is to perform several operations in one step. Consider that we want to perform the operations for n and let n_L and n_H be the least significant two bits and the remaining bits of n. In other words, $n = 4n_H + n_L$ holds. Clearly, the value of n_L is either 00, 01, 10, or 11. We can perform the several operations for n based on n_L as follows:

- $n_L = 00$: Since two even operations are applied, the resulting number is n_H.
- $n_L = 01$: First, odd operation is applied and the resulting number is $(4n_H + 1) \cdot 3 + 1 = 12n_H + 4$. After that, two even operations are applied, and we have $3n_H + 1$.
- $n_L = 10$: First, even operation is performed and we have $2n_H + 1$. Second, odd operation is applied and we have $(2n_H + 1) \cdot 3 + 1 = 6n_H + 4$. Finally, by even operation, the value is $3n_H + 2$.
- $n_L = 11$: First, odd operation is applied and we have $(4n_H + 3) \cdot 3 + 1 = 12n_H + 10$. Second, by even operation, the value is $6n_H + 5$. Again, odd operation is performed and we have $(6n_H + 5) \cdot 3 + 1 = 18n_H + 16$. Finally, by even operation, we have $9n_H + 8$.

For example, if $n_L = 11$ then we can obtain $9n_H + 8$ by applying 4 operations, odd, even, odd, and even operations in turn. Let B and C be tables as follows:

	B	C
00	1	0
01	3	1
10	3	2
11	9	8

Using these tables, we can perform the following table operation, which emulates several odd and even operations:

table operation For least significant two bits n_L and the remaining most significant bits n_H of the value, the new value is $B[n_L] \cdot n_H + C[n_L]$.

Let us extend the table operation for least significant two bits to d bits. For an integer $n \geq 2^d$, let n_L and n_H be the least significant d bits, that is, $n = 2^d n_H + n_L$. We call d is *the base bits*. Suppose that, the even or odd operations are repeatedly performed on $n = 2^d n_H + n_L$. We use two integers a and b such that $n = b \cdot n_H + c$ to denote the current value of n. Initially, $b = 2^d$ and $c = n_L$. We repeatedly perform the following rules for b and c.

even rule If both b and c are even, then divide them by two.

odd rule If b is odd, then triple b, and triple c and add one.

These two rules are applied until no more rules can be applied, that is, until b is odd and c is even. It should be clear that, even and odd rules correspond to even and odd operations of the Collatz conjecture. If i even rules and j odd rules applied, then the value of b is $2^{d-i}3^j$. Thus, exactly d even rules are applied until the termination. After the termination, we can determine the value of elements in tables B and C such that $B[n_L] = b$ and $C[n_L] = c$. Using tables B and C, we can perform the table operation for d bits n_L, which involves d even operations and zero or more odd operations. In this way, we can accelerate the operation of the Collatz conjecture. In our previous paper [12], we have implemented for various numbers of bits of n_L. Our implementation results show that the performance is well balanced when the number of bits of n_L is 10.

The third technique to accelerate the verification of the Collatz conjecture is to skip numbers n such that we can guarantee that the resulting number is less than n after the table operation. For example, suppose we are using two bit table and $n_H > 0$. If $n_L = 00$ then the resulting value is n_H, which is less than n. Thus, we can skip the table operation for n if $n_L = 00$. If $n_L = 01$ then the resulting value is $3n_H + 1$, which is always less than $n = 4n_H + 1$, and we can skip the table operation. Similarly, if $n_L = 10$ then we can skip the table operation. On the other hand $n_L = 11$

Table I
PERFORMANCE OF COPROCESSORS OF THE PARALLEL IMPLEMENTATION AND THE SEQUENTIAL PIPELINE IMPLEMENTATION

	Parallel implementation [11]	Sequential Pipeline implementation (this paper)
Used Embedded blocks	4 multipliers	1 DSP48E block
Clock Frequency	49.520MHz	280.604MHz
Performance	2.47×10^8 numbers/s	3.88×10^8 numbers/s
Supported bits	78 bits	112 bits
Overflow probability	2.88×10^{-5}	1.33×10^{-15}

then the resulting value is $9n_H + 8$, which is always larger than n. Therefore, the Collatz conjecture is guaranteed to be true whenever $n_L \neq 11$, because it has been verified true for numbers less than n. Consequently, we need to execute the table operation for number n such that $n_L = 11$.

We can extend this idea for general case. For least significant d bits n_L, we say that n_L is not *mandatory* if the value of b is less than 2^d at some moment while even and odd rules are repeatedly applied. We can skip the verification for non mandatory n_L. The reason is as follows: Consider that for number n, we are applying even and odd rules. Initially, $b = 2^d$ and $c \leq 2^d - 1$ hold. Thus, while even and odd rules are applied, $b > c$ always hold. Suppose that $b \leq 2^d - 1$ holds at some moment while the rules are applied. Then, the current value of n is

$$bn_H + b < bn_H + b \leq (2^d - 1)n_H + b = 2^d n_H < n.$$

It follows that, the value is less than n when the corresponding even and odd operations are applied. Therefore, we can omit the verification for numbers that have no mandatory least significant bits.

For least significant d bit number, we use table S to store the mandatory least significant bits. Let s_d be the number of such mandatory least significant bits. Using these tables, we can write a verification algorithm as follows:

for $m_H \leftarrow 1$ **to** $+\infty$ **do**
  for $i \leftarrow 0$ **to** $s_d - 1$ **do**
    begin
      $m_L \leftarrow S[i]$;
      $n \leftarrow m \leftarrow 2^d m_H + m_L$;
      while($n \geq m$) **do**
        begin
          Let n_L be the least significant d bits and
            n_H be the remaining bits.
          $n \leftarrow B[n_L] \cdot n_H + C[n_L]$;
        end
    end

For the benefit of readers, we show B, C, and S for 4 base bits in Table II. From $s_4 = 3$, we have 3 mandatory least significant bits out of 16.

For the reader's benefit, Table III shows the necessary word size and necessary total bits for each of tables B and C for each base bit. It also shows the expected number of odd/even operations included in one step operation $n \leftarrow B[n_L] \cdot n_H + C[n_L]$. Table IV shows the size of table S. It further shows the ratio of the mandatory numbers over all numbers. Later, we will use two 36k-bit block RAM for tables B and C, and table S, we set base bit 10 for tables B and C, and base bit 15 for table S. Thus, in our implementation, one operation $n \leftarrow B[n_L] \cdot n_H + C[n_L]$ corresponds to expected 15 odd/even operations. Also, we skip approximately 96% of non-mandatory numbers.

Table II
TABLES B, C, AND S FOR LEAST SIGNIFICANT 4 BITS.

	A	B	S
0000	1	0	0111
0001	9	1	1011
0010	9	2	1111
0011	9	2	-
0100	3	1	-
0101	3	1	-
0110	9	4	-
0111	27	13	-
1000	3	2	-
1001	27	17	-
1010	3	2	-
1011	27	20	-
1100	9	8	-
1101	9	8	-
1110	27	26	-
1111	81	80	-

Table III
THE SIZE OF TABLES B AND C

base bit	words	word size	total bits	operation
4	16	7	112	6.0
5	32	8	256	7.5
6	64	10	640	9.0
7	128	12	1536	10.5
8	256	13	3328	12.0
9	512	15	7680	13.5
10	1k	16	16k	15.0
11	2k	18	36k	16.5
12	4k	20	80k	18.0
13	8k	21	168k	19.5
14	16k	23	368k	21.0
15	32k	24	768k	22.5
16	64k	26	1664k	24.0

Table IV
THE SIZE OF TABLES S

base bit	words	word size	total bits	ratio
4	3	4	12	0.1875
5	4	5	20	0.1250
6	8	6	48	0.1250
7	13	7	91	0.1016
8	19	8	152	0.0742
9	38	9	342	0.0742
10	64	10	640	0.0625
11	128	11	1408	0.0625
12	226	12	2712	0.0552
13	367	13	4771	0.0448
14	734	14	10276	0.0448
15	1295	15	19425	0.0395
16	2113	16	33808	0.0322

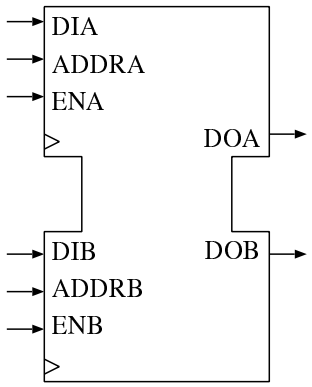

Figure 3. A dual-port block RAM

III. AN IMPLEMENTATION OF THE COMPUTATION $B[n_L] \cdot n_H + C[n_L]$ USING A BLOCK RAM AND DSP BLOCK

This section shows how the block RAM and the DSP block is used to implement the computation $B[n_L] \cdot n_H + C[n_L]$. For the reader's benefits, we first review the function of embedded block RAM and embedded DSP block.

Most FPGAs have embedded dual-port block RAMs as memory blocks. Figure 3 illustrates a dual-port block RAM. It has two ports A and B for which read/write operations to different address can be done in the same time. For example, Xilinx Virtex-5 family FPGAs have 36kbit block RAMs, each of which can be configured as $32k \times 1$, $16k \times 2$, $8k \times 4$, $4k \times 9$, $2k \times 18$, $1k \times 36$ memory. See [14] for the details of the Xilinx Virtex-5 block RAM.

Further, several FPGAs have embedded DSP blocks that can perform multiplication and addition in high frequency. For example, Xilinx Virtex-5 family FPGAs have DSP48E blocks, which can compute multiplication, addition, logic operations, overflow detection, etc. Figure 4 shows a part of the block diagram of Xilinx DSP48E block. One DSP48E block includes three input ports A, B, and C, and one output port P. In the figure, P_L and P_H denote the least significant 17 bits of P and the remaining bits. Using the DSP48E block, operations $P \leftarrow A \cdot B + C$ or $P \leftarrow A \cdot B + P_H$ may be performed in more than 500MHz while the same operations take less than 50MHz without using the DSP48E block in the FPGA. See [15] for the details of Xilinx DSP48E block.

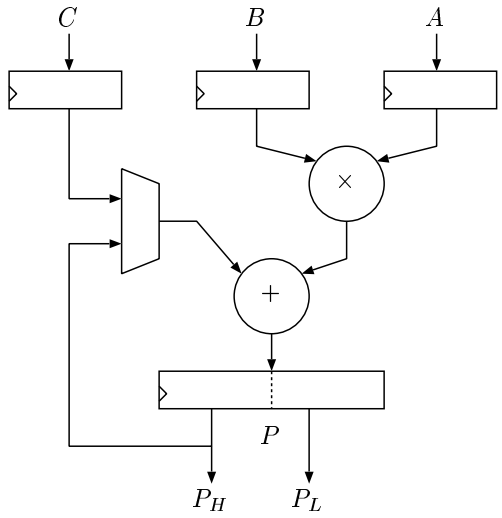

Figure 4. A part of the block diagram of Xilinx DSP48E block

In our implementation, we use 36kbit block RAMs (Figure 3.) One block RAM is used to implement the tables B and C with base bit 10. Also, one block is used for the table S with base bit 15. As shown in Table III, the values of B and C with base bit 10 are at most 16 bit. Thus, a 36kbit block RAM is configured as a $1k \times 36$ memory to store the values. More specifically, for each i $(0 \leq i \leq 2^{10})$, the values of 32bit $C[i] : B[i]$ is stored in address i of the block RAM. Using the block RAM, the values of $B[n_L]$ and $C[n_L]$ can be computed. Also, since the table S with base bit 15 needs $2k \times 15$ words from Table IV, we use one 36kbit block RAM to store it.

A DSP block is used to compute the value of $B[n_L] \cdot n_H + C[n_L]$. As illustrated in Figure 4, a DSP block can compute $P \leftarrow A \cdot B + C$ or $P \leftarrow A \cdot B + P_H$. In the Xilinx DSP48E block, A and B can have up to 25 bits and 18 bits, respectively, and both C and P can have 48 bits. Also, it supports unsigned 23×17-bit multiplication and 48-bit addition. Output P is separated into 31-bit P_H and 17-bit P_L.

We implement the computation of $n \leftarrow B[n_L] \cdot n_H + C[n_L]$ as follows. The values of $B[n_L]$ and $C[n_L]$ are given to ports B and C, respectively. Let n_H is partitioned into k 17-bit unsigned integers $n_0, n_1, \ldots, n_{k-1}$ such that $n_H = n_0 \cdot 2^{0 \cdot 17} + n_1 \cdot 2^{1 \cdot 17} + \cdots + n_{k-1} \cdot 2^{(k-1) \cdot 17}$. Similarly, the resulting value p is partitioned into $k+1$ 17-bit unsigned integers $p_0, p_1, \ldots, p_k$ The values of $n_0, n_1, \ldots, n_{k-1}$ are

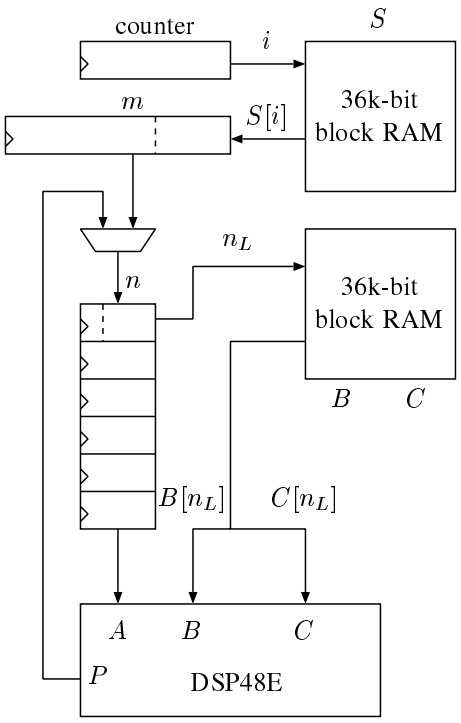

Figure 5. The architecture of a coprocessor

given to port A in turn. Then, the resulting values of $p_0, p_1, \ldots, p_k$ are computed, and these values can be obtained from port P_L in turn. The key idea is to compute the value of p from the least significant digit. Register P_L is used to store the carry. The details are spelled out as follows:

1 $P \leftarrow B[n_L] \cdot n_0 + C[n_L]$;
for $i \leftarrow 0$ to $k-2$
2 $p_i \leftarrow P_L$, $P \leftarrow B[n_L] \cdot n_{i+1} + P_H$;
3 $p_{k-1} \leftarrow P_L$;$p_k \leftarrow P_H$;

In this hardware algorithm, each of the lines 1, 2, and 3 can be computed in one clock cycle. In this way, the values of $n_0, n_1, \ldots, n_k$ are computed.

To achieve the high clock frequency, we must use internal registers to partition the combination circuit in DSP block and perform the computation in pipeline fashion. For this purpose, we have used internal registers such that the computation is performed in four stage pipeline. Also, we have used eight 17-bit registers to store p_0, p_1, ..., p_7 as illustrated in Figure 2.

Figure 5 illustrates the architecture of a coprocessor. One block RAM is used to store the values of S with base bit 15. The other block RAM is used to store the values of B and C with base bit 10. Thus, the table S has 1,295 mandatory least significant bits, a counter is used to output numbers i from 0 to 1,294. One block RAM is used to output $S[i]$, which determine the least significant 15 bits of m. The value of m is transferred to six 17-bit registers, which store the current value of n. The least significant 15 bits of n, n_L is given to the block RAM for tables B and C. The block RAM outputs the values of $B[n_L]$ and $B[n_L]$, which are given to ports B and C of the DSP48E block.

IV. The probability of overflow

Let us evaluate how often we have the overflow. Suppose that the even and the odd operations are performed for a 64-bit number n. Note that $2^{63} \leq n < 2^{64}$ holds. Let $M(n)$ denote the maximum number until n produces, for the first time, a number less than n during the operations. For example, $M(3) = 16$ holds. Suppose that we use b-bit architecture for the even and the odd operations. The verification of n is overflow if $M(n) \geq 2^b$. Let $V(b)$ be the set of such number n, that is $V(b) = \{n \mid 2^{63} \leq n < 2^{64} \text{ and } M(n) \geq 2^b\}$. Then, the ratio $R(b)$ of the overflow of 64-bit numbers on b-bit architecture is

$$R(b) = \frac{|V(b)|}{2^{63}}.$$

Figure 6 shows the ratio $R(b)$ of overflow for a 64-bit integer on the b-bit architecture. The ratio $R(b)$ for $b = 66, 67, \ldots, 84$ generating a 64-bit number 10^8 times at random, and seeing if $M(n) \geq 2^b$ [11]. Using the results, we expect the ratio $R(b)$ for $b > 84$. This figure shows that the ratio $R(b)$ rapidly decreases by increasing the value of b. Recall that the parameter $b = 112$ is used in our coprocessors. From the figure, we can see $R(112) = 1.33 \times 10^{-15}$, which is small enough.

V. Experimental results

We have implemented and evaluated the performance of our sequential pipeline implementation on Xilinx Virtex-5 family FPGA (XC5VSX50T-1FF1136), which has 8,160 slices, 288 DSP48E blocks, and 132 36k-bit block RAMs. Table V shows the result of synthesis of our implementation. For the purpose of showing the goodness of our sequential pipeline implementation, the table also shows the result of synthesis of our previous implementation, the parallel implementation [11]. We have implemented and evaluated the performance of our previous implementation on Xilinx Virtex-2 family FPGA (XC2V3000-4FG676), which has 14,336 slices, 96 multipliers, and 96 18k-bit block RAMS. Since the structure of each FPGA is different, the size of each circuit is not comparable. However, compared with the number of available slices in each FPGA, each size of both implementations is small enough. Therefore, several dozen coprocessors can be implemented in an FPGA and they can verify the conjecture in parallel.

We have evaluated the computing time of the FPGA implementations by verifying the Collatz conjecture for the 64-bit numbers. For this purpose, we have randomly generated

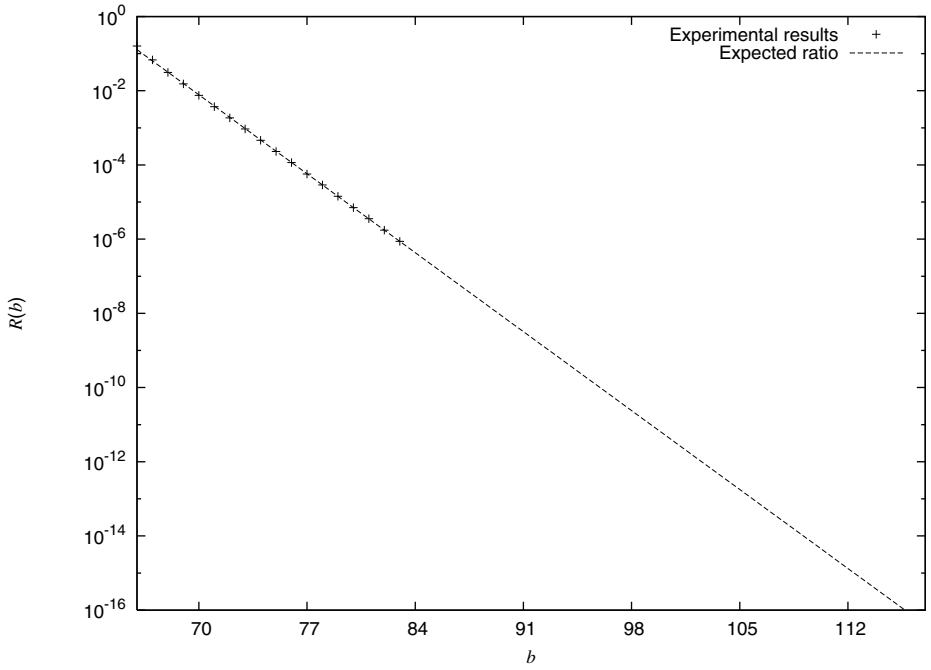

Figure 6. The ratio $R(b)$ of overflow for a 64-bit integer on the b-bit architecture

Table V
THE RESULTS OF SYNTHESIS OF OUR IMPLEMENTATION

	Parallel implementation [11]	Sequential pipeline implementation(this paper)
Target Device	XC2V3000-4FG676	XC5VSX50T-1FF1136
Used Embedded Block	4 multipliers	1 DSP48E block
Used Slices	282	113
Used RAMs	4 18k-bit block RAMs	2 36k-bit block RAMs
Maximum Clock Frequency	49.520MHz	280.604MHz

240 32-bit numbers M at random. For each generated 32-bit random number M, the FPGA implementations verified 2^{32} numbers in the range of $[M \cdot 2^{32}, (M+1) \cdot 2^{32} - 1]$. Therefore, they verified verified $240 \times 2^{32} \approx 1.0 \times 10^{12}$ 64-bit numbers in total. Table VI shows the experimental results. Our new pipeline implementation is approximately 1.5 times faster than the previous implementation. Also, our previous implementation has 5,656,255 overflows in 10^{12} 64-bit numbers, while our new implementation has detected no over flow numbers. In our experiment of verifying 10^{12} 64-bit numbers, no interim number has more than 100 bits. Recall that our new implementation and previous implementation support 112-bit and 78-bit interim numbers. Thus, our previous implementation detects many overflowed interim numbers, while our new implementation has no overflow numbers. Actually, as shown in the previous section, $R(112) = 1.33 \times 10^{-15}$. Thus, we have expected 1.33×10^{-3} overflow interim numbers during the verification of 10^{12} 64-bit numbers.

VI. CONCLUSIONS

We have presented an efficient implementation of a coprocessor that performs the exhaustive search to verify the Collatz conjecture using a DSP48E Xilinx Virtex-5 blocks, each of which contains one multiplier and one adder.

We have implemented our coprocessor on the Virtex-5 family FPGA XC5VSX50T-1. The experimental results show that it can verify 3.88×10^8 64-bit numbers per second. Since the size of the coprocessor is small enough, several dozen coprocessors can be implemented in an FPGA. Therefore, they can work in parallel and verify the conjecture much faster.

REFERENCES

[1] J. L. Bordim, Y. Ito, and K. Nakano, "Accelerating the CKY parsing using FPGAs," *IEICE Transactions on Information and Systems*, vol. E86-D, no. 5, pp. 803–810, May 2003.

[2] ——, "Instance-specific solutions to accelerate the CKY parsing for large context-free grammars," *International Journal on Foundations of Computer Science*, pp. 403–416, 2004.

Table VI
THE COMPUTING TIME FOR VERIFYING COLLATZ CONJECTURE

	Parallel implementation [11]	Sequential Pipeline implementation(this paper)
Clock frequency	49.520MHz	280.604MHz
Computing time for verifying 10^{12} numbers	4,174.63 sec	2,654.63 sec
Th number of verified numbers per second	2.47×10^8	3.88×10^8
The number of overflowed interim numbers	5,656,255	0

[3] K. Nakano and E. Takamichi, "An image retrieval system using FPGAs," *IEICE Transactions on Information and Systems*, vol. E86-D, no. 5, pp. 811–818, May 2003.

[4] K. Nakano and K. Wada, "Integer summing algorithms on reconfigurable meshes," *Theoretical Computer Science*, vol. 197, pp. 57–77, 1998.

[5] K. Nakano and Y. Yamagishi, "Hardware n choose k counters with applications to the partial exhaustive search," *IEICE Trans. on Information & Systems*, 2005.

[6] T. Oliveira e Silva, "Maximum excursion and stopping time record-holders for the $3x+1$ problem: Computational results," *Mathematics of Computation*, vol. 68, no. 225, pp. 371–384, Jan. 1999, up to date computational results can be found at http://www.ieeta.pt/˜tos/3x+1.html.

[7] J. C. Lagarias, "The 3x+1 problem and its generalizations," *The American Mathematical Monthly*, vol. 92, no. 1, pp. 3–23, 1985.

[8] E. W. Weisstein, "Collatz problem," From MathWorld–A Wolfram Web Resource. http://mathworld.wolfram.com/CollatzProblem.html.

[9] E. Roosendaal, "On the 3x + 1 problem," http://www.ericr.nl/wondrous/index.html.

[10] T. Oliveira e Silva, "Computational verification of the 3x+1 conjecture," http://www.ieeta.pt/˜tos/3x+1.html.

[11] Y. Ito and K. Nakano, "A hardware-software cooperative approach for the exhaustive verification of the collatz cojecture," in *Proc. of International Symposium on Parallel and Distributed Processing with Applications*, 2009, pp. 63–70.

[12] F. An and K. Nakano, "An architecture for veryfing collatz conjecture using an fpga," in *Proc. of the International Conference on Applications and Principles of Informatin Science*, 2009, pp. 375–378.

[13] S. Ichikawa and N. Kobayashi, "Preliminary study of custom computing hardware for the 3x+1 problem," in *Proc. of IEEE TENCON 2004*, 2004, pp. 387–390.

[14] Xilinx Inc., *Virtex-5 FPGA Users Guide*, 2009.

[15] ——, *Virtex-5 DSP48E Users Guide*, 2007.